Power Query ETL & Data Cleaning Documentation

Step-by-Step Technical Guide for the Egyptian Real Estate Dataset Pipeline

Project: DEPI Graduation Project	**Track:** Data Analytics & BI	**Environment:** Microsoft Power BI	**Language:** Power Query (M)
Source File: final (2).csv	**Raw Records:** 249,999 Rows	**Initial Columns:** 28 Attributes	**Final Schema:** 31 Cleaned Attributes

1. Pipeline Architecture & Executive Overview

This technical document provides a comprehensive, step-by-step architectural breakdown of the data cleaning and transformation pipeline developed in Microsoft Power Query (M formula language) for the Egyptian Real Estate dataset. The underlying dataset captures 249,999 raw property listings scraped across diverse online real estate portals. Prior to analytical modeling, the raw records suffered from typical web-scraping and multi-source anomalies: re-scraped duplicate records, irregular whitespace padding, non-standardized categorical terms, human typos in property types and governorate names, shorthand currency text ('15M', 'EGP', 'NULL'), mixed datetime formats, decimal-comma coordinates, combined location strings, and messy amenity delimiters.

The ETL pipeline implements 16 sequential transformations in the Power Query Advanced Editor. It cleanses messy values, performs custom feature engineering (extracting a 3-tier geographic hierarchy and quantifying luxury amenities), reorganizes attributes into logical business domains, and strictly locks schema data types. The resulting model yields 246,077 clean, unique records across 31 typed columns, perfectly primed for high-performance Power BI reporting.

Summary of ETL Pipeline Transformation Stages

Step #	Applied Step Name	Target Columns / Scope	Key Transformation Objective
01	**Source, Promote Headers & Deduplication**	File level & listing_id	Read UTF-8 CSV, promote headers, and remove 3,922 duplicate listings.
02	**Trimmed Text (All Columns)**	17 text and date attributes	Strip all leading, trailing, and residual whitespace characters.
03	**Standardized Category & Price Period**	category, price_period	Harmonize into binary 'Rent'/'Sale' and 'Monthly'/'Total' values.
04	**Standardized Status (Clean Typos)**	status	Correct typos (e.g., 'solld', 'rentd') into 4 clean business states.

Step #	Applied Step Name	Target Columns / Scope	Key Transformation Objective
05	**Standardized Property Types**	property_type	Consolidate 12 canonical property typologies with unified proper casing.
06	**Standardized Cities (Governorates)**	city	Harmonize 25+ Egyptian governorates across transliterations and typos.
07	**Cleaned Price (EGP & Millions)**	price_egp	Remove 'EGP'/'NULL', multiply 'M' suffixes by 1,000,000 to numeric values.
08	**Cleaned Sold Price (EGP & Millions)**	sold_price	Apply multiplier logic while preserving strict nulls for unsold properties.
09	**Parsed Dates (Listed & Sale Dates)**	listed_date, sale_date	Handle ISO timestamps, US date formats, and hyphens into true Date types.
10	**Cleaned Coordinates (lat & lon)**	lat, lon	Fix comma decimals and integer scaling corruption for valid Egyptian geo pins.
11	**Cleaned Image Count & Booleans**	image_count, new_const, verified	Isolate image numbers and map strings ('yes', '1') to true booleans.
12	**Location Hierarchy Extraction**	compound_neighborhood, district	Feature engineer granular Compound and District levels from raw text.
13	**Standardized Amenities & Count**	amenities, amenities_count	Standardize delimiters to commas and calculate total property amenities.
14	**Capitalized Broker & General Text**	broker fields, payment_method	Format names, emails, furnishing, and map payment methods cleanly.
15	**Reordered Columns**	All 31 attributes	Group columns into logical business clusters for optimal report design.
16	**Final Schema Data Type Assignment**	All 31 attributes	Explicitly lock data types for VertiPaq engine compression and DAX speed.

2. Detailed Technical Step-by-Step Documentation

This section details each individual transformation in the Power Query script. For each stage, the technical problem in the raw data is explained, followed by the M transformation logic, the exact M code snippet, and the analytical impact on data modeling in Power BI.

Step 1: Data Ingestion, Header Promotion, and Primary Key Deduplication

Applied Power Query Steps: Source, #"Promoted Headers", #"Removed Duplicates(listing id)"

Problem in Raw Data: The raw file 'final (2).csv' contains 249,999 raw property listings scraped from various online portals. Scraped real estate repositories frequently suffer from repeated scraping cycles, resulting in identical duplicate records. Furthermore, importing raw CSV files requires strict encoding handling to preserve Arabic geographical terms (such as Cairo district names and compound titles) without character corruption.

Transformation Logic & Implementation: 1. Data Ingestion: The file is loaded via Csv.Document using File.Contents. Crucially, Encoding=65001 (UTF-8) is explicitly specified to guarantee full Unicode support for Arabic characters. The column count is strictly bounded to 28 columns with QuoteStyle.Csv to prevent misplaced commas inside quoted description strings from breaking the row structure.
2. Header Promotion: Table.PromoteHeaders elevates the initial scalar text row into official column names, establishing the base table schema.
3. Primary Key Deduplication: Table.Distinct evaluates the primary identifier column 'listing_id'. Exactly 3,922 duplicate records are purged, reducing the active dataset from 249,999 down to 246,077 strictly unique listings. This eliminates inflated inventory counts and guarantees a valid 1:1 primary key for dimensional modeling in Power BI.

```
// 1. استيراد الملف وترقية العناوين وحذف التكرار
Source = Csv.Document(File.Contents("D:\DEPI\Final Project\final (2).csv"),[Delimiter=",",
Columns=28, Encoding=65001, QuoteStyle=QuoteStyle.Csv]),
#"Promoted Headers" = Table.PromoteHeaders(Source, [PromoteAllScalars=true]),
#"Removed Duplicates(listing id)" = Table.Distinct(#"Promoted Headers", {"listing_id"}),
```

> **Key Analytical Takeaway:** Data Uniqueness: Removing 3,922 duplicate listings ensures that aggregations (such as Total Available Real Estate Value and Average Price per Meter) are never double-counted.

Step 2: Universal Whitespace Stripping Across Text Attributes

Applied Power Query Steps: #"Trimmed Text(all columns)"

Problem in Raw Data: Web scraping artifacts, HTML tags, and manual user inputs regularly inject invisible leading spaces, trailing whitespace, or double tabs into text and unparsed numeric/date fields. For example, ' Cairo' vs 'Cairo ' would be interpreted by Power BI as two distinct categorical values in slicers, producing fragmented aggregations and broken relationship joins.

Transformation Logic & Implementation: The Table.TransformColumns function applies the built-in Text.Trim operation across 17 text, numeric-string, and date-string columns simultaneously. This purges all invisible whitespace characters from the front and back of each text cell across: category, listing_type, property_type, price_period, city, location_full, status, furnished, listing_level, completion_status, payment_method, broker_name, broker_email, price_egp, sold_price, listed_date, and sale_date.

Applying universal trimming early in the ETL pipeline guarantees that subsequent pattern matching (e.g., checking if a string ends with 'M' or matching city names) executes reliably without silent failures caused by stray whitespace.

```
// 2. ازالة المسافات الزائدة من كل النصوص
#"Trimmed Text(all columns)" = Table.TransformColumns(#"Removed Duplicates(listing id)", {
    {"category", Text.Trim, type text},
    {"listing_type", Text.Trim, type text},
    {"property_type", Text.Trim, type text},
    {"price_period", Text.Trim, type text},
    {"city", Text.Trim, type text},
    {"location_full", Text.Trim, type text},
    {"status", Text.Trim, type text},
    {"furnished", Text.Trim, type text},
    {"listing_level", Text.Trim, type text},
    {"completion_status", Text.Trim, type text},
    {"payment_method", Text.Trim, type text},
    {"broker_name", Text.Trim, type text},
    {"broker_email", Text.Trim, type text},
    {"price_egp", Text.Trim, type text},
    {"sold_price", Text.Trim, type text},
    {"listed_date", Text.Trim, type text},
    {"sale_date", Text.Trim, type text}
}),
```

> **Key Analytical Takeaway:** Data Hygiene: Text.Trim prevents categorical fragmentation, ensuring clean slicers and unified group-by summaries in reporting.

Step 3: Standardization of Listing Category & Pricing Frequency

Applied Power Query Steps: #"Standardized Category(Rent/Sale)", #"Standardized Price Period(Monthly/Total)"

Problem in Raw Data: The raw data contained overlapping and inconsistent descriptors for transaction categories ('rent', 'rental', 'for rent', 'sale', 'buy') and pricing periodicity ('monthly', 'month', total rental sums). Without harmonization, financial comparisons are impossible because recurring monthly rentals would be directly aggregated alongside multi-million-pound property purchases.

Transformation Logic & Implementation: 1. Category Standardization: An inline lambda converts the 'category' value to lowercase using Text.Lower(_). If the string matches 'rent', 'rental', or 'for rent', it is standardized to 'Rent'. All other variations (including 'sale', 'buy', and 'for sale') are normalized to 'Sale'.
2. Price Period Standardization: The 'price_period' field is similarly evaluated. Lowercase representations of 'monthly' and 'month' are mapped to 'Monthly'. All other records (such as property sale prices and one-off long-term contracts) are categorized as 'Total'.
This clear distinction allows DAX measures to filter rental cashflows independently from asset purchase valuations.

```
// 3. توحيد التصنيفات وفترات الأسعار
#"Standardized Category(Rent/Sale)" = Table.TransformColumns(#"Trimmed Text(all columns)", {
    {"category", each
        let s = Text.Lower(_) in
```

```
        if List.Contains({"rent", "rental", "for rent"}, s) then "Rent"
        else "Sale",
        type text}
}),

#"Standardized Price Period(Monthly/Total)" = Table.TransformColumns(#"Standardized
Category(Rent/Sale)", {
    {"price_period", each
        let s = Text.Lower(_) in
        if List.Contains({"monthly", "month"}, s) then "Monthly"
        else "Total",
        type text}
}),
```

> **Key Analytical Takeaway:** Business Alignment: Establishes clear financial boundaries so periodic rental yields are not erroneously summed with capital purchase prices.

Step 4: Property Operational Status Cleaning & Typo Resolution

Applied Power Query Steps: #"Standardized Status(Clean Typos)"

Problem in Raw Data: The 'status' attribute suffered from typographical errors and inconsistent naming conventions originating from varied source systems. Entries included 'avaliable', 'solld', 'rentd', 'active', 'for sale', and 'for rent'. Without correction, filtering for active inventory or calculating total historical sales would return inaccurate results.

Transformation Logic & Implementation: The transformation inspects the lowercase representation of 'status' and consolidates all records into four mutually exclusive operational states:
• Available: Maps active listings including 'available', 'avaliable' (typo), 'active', 'for sale', and 'for rent'.
• Sold: Maps finalized purchase transactions including 'sold' and 'solld' (typo).
• Rented: Maps executed lease contracts including 'rented' and 'rentd' (typo).
• Off Market: Default fallback for expired, withdrawn, or suspended listings.
This guarantees that dashboard viewers can segment active properties versus closed historical transactions with 100% data integrity.

```
// 4. تصحيح حالات العقارات وتوحيدها
#"Standardized Status(Clean Typos)" = Table.TransformColumns(#"Standardized Price
Period(Monthly/Total)", {
    {"status", each
        let s = Text.Lower(_) in
        if List.Contains({"available", "avaliable", "active", "for sale", "for rent"}, s) then
"Available"
        else if List.Contains({"sold", "solld"}, s) then "Sold"
        else if List.Contains({"rented", "rentd"}, s) then "Rented"
        else "Off Market",
        type text}
}),
```

> **Key Analytical Takeaway:** Operational Integrity: Cleanses user typos and consolidates statuses into 4 distinct business buckets: Available, Sold, Rented, and Off Market.

Step 5: Property Type Harmonization & Structural Typology Mapping

Applied Power Query Steps: #"Standardized Property Types"

Problem in Raw Data: Property classifications contained varied spelling variants, informal colloquial abbreviations, and English transliterations (e.g., 'apt', 'apartement', 'appartment', 'vila', 'dublex', 'chalett', 'i villa'). This fragmentation resulted in over 30 distinct property classifications for what are fundamentally 12 standard Egyptian housing types.

Transformation Logic & Implementation: An intelligent conditional mapping block evaluates the lowercase string and re-aligns all records into 12 canonical Egyptian property typologies:
1. Apartment: 'apartment', 'apt', 'apartement', 'appartment'
2. Villa: 'villa', 'vila'
3. Townhouse: 'townhouse', 'town house'
4. Twin House: 'twin house', 'twinhouse', 'twin villa'
5. Duplex: 'duplex', 'dublex'
6. Penthouse: 'penthouse', 'pent house'
7. Chalet: 'chalet', 'chalett' (coastal vacation units)
8. Hotel Apartment: 'hotel apartment', 'hotel apt'
9. iVilla: 'ivilla', 'i villa', 'i-villa' (trademark Mountain View typology)
10. Whole Building: 'whole building', 'building'
11. Land: 'land', 'plot'
12. Roof: 'roof'

Any unmapped niche category falls back gracefully to Text.Proper(t) to ensure clean title casing.

```
// 5. توحيد أنواع العقارات (Apartment, Villa, Duplex, ...)
#"Standardized Property Types" = Table.TransformColumns(#"Standardized Status(Clean Typos)", {
    {"property_type", each
        let t = Text.Lower(_) in
        if List.Contains({"apartment", "apt", "apartement", "appartment"}, t) then "Apartment"
        else if List.Contains({"villa", "vila"}, t) then "Villa"
        else if List.Contains({"townhouse", "town house"}, t) then "Townhouse"
        else if List.Contains({"twin house", "twinhouse", "twin villa"}, t) then "Twin House"
        else if List.Contains({"duplex", "dublex"}, t) then "Duplex"
        else if List.Contains({"penthouse", "pent house"}, t) then "Penthouse"
        else if List.Contains({"chalet", "chalett"}, t) then "Chalet"
        else if List.Contains({"hotel apartment", "hotel apt"}, t) then "Hotel Apartment"
        else if List.Contains({"ivilla", "i villa", "i-villa"}, t) then "iVilla"
        else if List.Contains({"whole building", "building"}, t) then "Whole Building"
        else if List.Contains({"land", "plot"}, t) then "Land"
        else if List.Contains({"roof"}, t) then "Roof"
        else Text.Proper(t),
        type text}
}),
```

> **Key Analytical Takeaway:** Market Specialization: Unifies Egyptian real estate models including regional specialties like 'iVilla' and North Coast 'Chalet' properties.

Step 6: Governorate & City Name Harmonization

Applied Power Query Steps: #"Standardized Cities(Governorates)"

Problem in Raw Data: Because real estate portals cater to multilingual audiences, the 'city' column contained a mixture of English names, French transliterations, and Arabic romanizations with and without the definite article 'Al/El' (e.g., 'Cairo' vs 'Al Qahira', 'Alexandria' vs 'Alex', 'Damietta' vs 'Demyat' / 'Domyat', 'Minya' vs 'El Minya' / 'Menia', 'Gharbia' vs 'Gharbeya').

Transformation Logic & Implementation: A comprehensive multi-branch dictionary standardizes 25+ Egyptian governorates and key geographical sectors:
• Major Metros: 'Cairo' ('cairo', 'al qahira'), 'Giza' ('giza', 'al giza', 'gizeh'), 'Alexandria' ('alexandria', 'alex').
• Delta & Canal: 'Dakahlia', 'Sharqia', 'Damietta', 'Qalyubia', 'Menoufia', 'Gharbia', 'Kafr El Sheikh', 'Beheira', 'Port Said', 'Suez', 'Ismailia'.
• Upper Egypt: 'Fayoum', 'Beni Suef', 'Minya', 'Assiut', 'Sohag', 'Qena', 'Luxor', 'Aswan'.
• Coastal & Frontier: 'Red Sea', 'South Sinai', 'North Sinai', 'North Coast', 'Matrouh', 'New Valley'.
• Unmapped locations default to Text.Proper(c).
This guarantees unified drill-downs and ensures Power BI's geographic mapping engine can accurately match cities to coordinates.

```
// 6. توحيد أسماء المحافظات
#"Standardized Cities(Governorates)" = Table.TransformColumns(#"Standardized Property Types", {
    {"city", each
        let c = Text.Lower(_) in
        if List.Contains({"cairo", "al qahira"}, c) then "Cairo"
        else if List.Contains({"giza", "al giza", "gizeh"}, c) then "Giza"
        else if List.Contains({"alexandria", "alex"}, c) then "Alexandria"
        else if List.Contains({"dakahlia", "al daqahlya", "daqahlia"}, c) then "Dakahlia"
        else if List.Contains({"sharqia", "sharkia", "al sharqia"}, c) then "Sharqia"
        else if List.Contains({"damietta", "demyat", "domyat"}, c) then "Damietta"
        else if List.Contains({"minya", "el minya", "menia"}, c) then "Minya"
        else if List.Contains({"qalyubia", "qalubia", "kalyubia"}, c) then "Qalyubia"
        else if List.Contains({"red sea", "redsea"}, c) then "Red Sea"
        else if List.Contains({"south sinai", "south sainai", "sinai"}, c) then "South Sinai"
        else if List.Contains({"north sinai"}, c) then "North Sinai"
        else if List.Contains({"north coast", "northcoast"}, c) then "North Coast"
        else if List.Contains({"suez"}, c) then "Suez"
        else if List.Contains({"fayoum", "faiyum"}, c) then "Fayoum"
        else if List.Contains({"beni suef", "bani suef"}, c) then "Beni Suef"
        else if List.Contains({"menoufia", "menofia", "monufia"}, c) then "Menoufia"
        else if List.Contains({"gharbia", "gharbeya"}, c) then "Gharbia"
        else if List.Contains({"kafr el sheikh", "kafr elsheikh"}, c) then "Kafr El Sheikh"
        else if List.Contains({"port said", "portsaid"}, c) then "Port Said"
        else if List.Contains({"qena", "qina"}, c) then "Qena"
```

```
        else if List.Contains({"sohag", "suhag"}, c) then "Sohag"
        else if List.Contains({"assiut", "asyut"}, c) then "Assiut"
        else if List.Contains({"beheira", "behira"}, c) then "Beheira"
        else if List.Contains({"matrouh"}, c) then "Matrouh"
        else if List.Contains({"aswan"}, c) then "Aswan"
        else if List.Contains({"luxor"}, c) then "Luxor"
        else if List.Contains({"ismailia", "ismalia"}, c) then "Ismailia"
        else if List.Contains({"new valley"}, c) then "New Valley"
        else Text.Proper(c),
        type text}
}),
```

> **Key Analytical Takeaway:** Geographic Consistency: Resolves Egyptian governorate transliterations, enabling accurate regional slicing and Power BI map pin rendering.

Step 7: Listing Price Cleaning & Shorthand Multiplier Expansion

Applied Power Query Steps: #"Cleaned Price(EGP & Millions)"

Problem in Raw Data: The 'price_egp' column was stored as unstructured text containing currency tags ('15,000,000 EGP'), missing values represented as text strings ('NULL', '-', ''), and human shorthand abbreviations such as '15M' or '3.5M EGP' representing 15,000,000 and 3,500,000 EGP respectively. Power BI cannot perform arithmetic aggregations or calculate averages on text columns.

Transformation Logic & Implementation: The transformation parses each raw value into a pure decimal number:
1. Null Check: Blank text, 'NULL', and '-' are converted to true null values.
2. 'M' Multiplier Detection: If the trimmed uppercase string ends with 'M', the substring prior to 'M' is extracted. Text characters ('E', 'G', 'P', commas, spaces) are stripped using Text.Remove. The remaining numeric string is parsed as a decimal and multiplied by 1,000,000.
3. Standard Numeric Parsing: For entries without 'M', all non-numeric currency characters are removed, and Number.FromText converts the clean string into an exact numeric value.
The column type is cast to 'type number', enabling mathematical KPI calculations.

```
// 7. تنظيف الأسعار (حذف EGP، وحساب حرف M الفواصل، بضربها في مليون)
#"Cleaned Price(EGP & Millions)" = Table.TransformColumns(#"Standardized Cities(Governorates)", {
    {"price_egp", each
        let
            str = Text.Upper(Text.Trim(Text.From(_)))
        in
            if str = "" or str = "NULL" or str = "-" then null
            else if Text.EndsWith(str, "M") then
                let
                    clean = Text.Trim(Text.Remove(Text.BeforeDelimiter(str, "M"), {"E", "G", "P", "
", ","})),
                    val = try Number.FromText(clean, "en-US") otherwise null
                in
                    if val <> null then val * 1000000 else null
```

```
            else
                let
                    clean = Text.Trim(Text.Remove(str, {"E", "G", "P", " ", ","})),
                    val = try Number.FromText(clean, "en-US") otherwise null
                in
                    val,
        type number}
}),
```

> **Key Analytical Takeaway:** Financial Accuracy: Automatically expands shorthand notations like '15M EGP' into 15,000,000, unlocking core revenue calculations.

Step 8: Transaction Sold Price Cleaning with Null Preservation

Applied Power Query Steps: #"Cleaned Sold Price(EGP & Millions)"

Problem in Raw Data: Similar to listing prices, the 'sold_price' column contained textual currency suffixes and million multipliers ('M'). However, unlike active listing prices, properties that are currently Available or Off Market do not have a closed sale transaction. It is critical that unsold listings remain strictly null rather than defaulting to 0 or arbitrary numbers, which would severely distort average sales metrics.

Transformation Logic & Implementation: The transformation implements defensive parsing:
1. If the input is null, it immediately outputs null.
2. If the string evaluates to empty, 'NULL', or '-', it explicitly returns null.
3. For properties that were actually sold or rented, the identical million-multiplier ('M' * 1,000,000) and currency character stripping logic is executed safely.
This ensures complete statistical validity: Average Realized Sale Price measures only closed sales without being dragged down by active or unsold listings.

```
// 8. تنظيف سعر البيع (للعقارات غير المباعة null مع الإبقاء على)
#"Cleaned Sold Price(EGP & Millions)" = Table.TransformColumns(#"Cleaned Price(EGP & Millions)", {
    {"sold_price", each
        if _ = null then null
        else
            let
                str = Text.Upper(Text.Trim(Text.From(_)))
            in
                if str = "" or str = "NULL" or str = "-" then null
                else if Text.EndsWith(str, "M") then
                    let
                        clean = Text.Trim(Text.Remove(Text.BeforeDelimiter(str, "M"), {"E", "G",
"P", " ", ","})),
                        val = try Number.FromText(clean, "en-US") otherwise null
                    in
                        if val <> null then val * 1000000 else null
                else
                    let
```

```
                        clean = Text.Trim(Text.Remove(str, {"E", "G", "P", " ", ","})),
                        val = try Number.FromText(clean, "en-US") otherwise null
                    in
                        val,
        type number}
}),
```

> **Key Analytical Takeaway:** Statistical Integrity: Preserves nulls for unsold properties, preventing zeros from skewing actual sales averages and discount rates.

Step 9: Robust Multi-Pattern Date Parsing (Listed & Sale Dates)

Applied Power Query Steps: #"Parsed Date(listed date)", #"Parsed Date(sale date)"

Problem in Raw Data: Because real estate listings originated from multiple database backends, timestamps were formatted inconsistently. Some rows used full ISO-8601 UTC timestamps (e.g., '2023-08-15T14:30:00Z'), others used standard US slash notation ('8/15/2023'), while others included trailing clock times ('2023-08-15 14:30:00') or text representations like 'NULL' and '-'. Standard Power Query date conversions would throw errors on mixed formats.

Transformation Logic & Implementation: The transformation establishes a defensive 3-tier parsing waterfall applied to both 'listed_date' and 'sale_date':
1. Null & Missing Value Handling: Null, empty strings, 'NULL', and '-' are safely converted to null.
2. ISO-8601 Timestamp Parsing: If the string contains 'T', DateTimeZone.From(str) extracts the timestamp and DateTime.Date isolates the date component. If that fails, it takes the first 10 characters (YYYY-MM-DD).
3. Standard Date Parsing: For non-ISO strings, Text.BeforeDelimiter isolates the date portion before any space/time component, and attempts parsing using en-US culture, falling back to invariant date converters.
Both columns are cleanly typed as 'type date', enabling Time Intelligence DAX functions (e.g., Days on Market = sale_date - listed_date).

```
// 9. تحويل تواريخ العرض والبيع لتاريخ موحد (والتواريخ ذات التوقيت والشرطات ISO يدعم)
#"Parsed Date(listed date)" = Table.TransformColumns(#"Cleaned Sold Price(EGP & Millions)", {
    {"listed_date", each
        if _ = null then null
        else
            let
                str = Text.Trim(Text.From(_))
            in
                if str = "" or str = "NULL" or str = "-" then null
                else if Text.Contains(str, "T") then
                    try DateTime.Date(DateTimeZone.From(str)) otherwise try
Date.FromText(Text.Start(str, 10)) otherwise null
                else
                    let
                        datePart = Text.BeforeDelimiter(str, " "),
                        d1 = try Date.FromText(datePart, [Culture = "en-US"]) otherwise null,
                        d2 = if d1 <> null then d1 else try Date.From(datePart, "en-US") otherwise
null,
```

```
                        d3 = if d2 <> null then d2 else try Date.From(datePart) otherwise null
                    in
                        d3,
        type date}
}),

#"Parsed Date(sale date)" = Table.TransformColumns(#"Parsed Date(listed date)", {
    {"sale_date", each
        if _ = null then null
        else
            let
                str = Text.Trim(Text.From(_))
            in
                if str = "" or str = "NULL" or str = "-" then null
                else if Text.Contains(str, "T") then
                    try DateTime.Date(DateTimeZone.From(str)) otherwise try
Date.FromText(Text.Start(str, 10)) otherwise null
                else
                    let
                        datePart = Text.BeforeDelimiter(str, " "),
                        d1 = try Date.FromText(datePart, [Culture = "en-US"]) otherwise null,
                        d2 = if d1 <> null then d1 else try Date.From(datePart, "en-US") otherwise
null,
                        d3 = if d2 <> null then d2 else try Date.From(datePart) otherwise null
                    in
                        d3,
        type date}
}),
```

> **Key Analytical Takeaway:** Time-Intelligence Ready: Unifies diverse timestamps into standard Date objects, unlocking Days on Market (DOM) and monthly sales velocity metrics.

Step 10: Geographic Coordinate Normalization & Decimal Correction

Applied Power Query Steps: #"Cleaned Coordinates(lat & lon)"

Problem in Raw Data: Geographic coordinates ('lat' and 'lon') contained severe regional data export corruption: (1) European decimal commas instead of points (e.g., '30,05' instead of '30.05'), and (2) missing decimal separators that scaled coordinates into massive integers (e.g., '3005128' or '3120549'). In Egypt, latitude must lie between 22° and 32° N, and longitude between 25° and 36° E. Scaled integers cause map visual components to crash or plot points in outer space.

Transformation Logic & Implementation: The M algorithm evaluates 'lat' and 'lon' independently:
1. If a comma is present with 2 or fewer digits following, Text.Replace converts the comma into a standard decimal point.
2. If commas represent missing decimal points or integer scaling (e.g., '30,051284' or digits without points), commas are stripped. If the resulting integer exceeds 1,000, it is dynamically scaled down by dividing by 10^(length - 2). For example, a 7-digit integer '3007858' is divided by 10^5 (100,000), producing exactly 30.07858.

3. Values are cast to 'type number'. This completely restores geographic coordinates to authentic Egyptian geospatial coordinates.

```
// 10. معالجة الإحداثيات (lat / lon) وضبط الفواصل العشرية
#"Cleaned Coordinates(lat & lon)" = Table.TransformColumns(#"Parsed Date(sale date)", {
    {"lat", each
        if _ = null then null
        else
            let
                s = Text.Trim(Text.From(_))
            in
                if s = "" then null
                else if Text.Contains(s, ",") then
                    let
                        commaCount = List.Count(Text.PositionOf(s, ",", Occurrence.All)),
                        res = if commaCount = 1 and Text.Length(Text.AfterDelimiter(s, ",")) <= 2
then
                            try Number.FromText(Text.Replace(s, ",", "."), "en-US") otherwise null
                        else
                            let
                                digitsOnly = Text.Remove(s, {","}),
                                rawNum = try Number.FromText(digitsOnly, "en-US") otherwise null
                            in
                                if rawNum = null then null
                                else if rawNum > 1000 then rawNum / Number.Power(10,
Text.Length(digitsOnly) - 2)
                                else rawNum
                    in
                        res
                else
                    try Number.FromText(s, "en-US") otherwise null,
        type number},
    {"lon", each
        if _ = null then null
        else
            let
                s = Text.Trim(Text.From(_))
            in
                if s = "" then null
                else if Text.Contains(s, ",") then
                    let
                        commaCount = List.Count(Text.PositionOf(s, ",", Occurrence.All)),
                        res = if commaCount = 1 and Text.Length(Text.AfterDelimiter(s, ",")) <= 2
then
                            try Number.FromText(Text.Replace(s, ",", "."), "en-US") otherwise null
                        else
                            let
                                digitsOnly = Text.Remove(s, {","}),
                                rawNum = try Number.FromText(digitsOnly, "en-US") otherwise null
                            in
                                if rawNum = null then null
```

```
                                else if rawNum > 1000 then rawNum / Number.Power(10,
Text.Length(digitsOnly) - 2)
                                else rawNum
                    in
                        res
                else
                    try Number.FromText(s, "en-US") otherwise null,
        type number}
}),
```

> **Key Analytical Takeaway:** Map-Ready Precision: Solves comma decimal anomalies and scales corrupt integers back to legitimate Egyptian coordinates (22°–32° N, 25°–36° E).

Step 11: Image Count Extraction & Boolean Flag Normalization

Applied Power Query Steps: #"Cleaned Text(image count)"

Problem in Raw Data: The 'image_count' column contained textual phrases like '12 photos' or '8 images' instead of pure integers. Concurrently, operational flags 'is_new_construction' and 'is_verified' contained mixed string variants ('true', 'yes', '1', 'false', '0', null) that could not be used natively as boolean filters in slicers or DAX logical expressions.

Transformation Logic & Implementation: 1. Image Count Parsing: Text.BeforeDelimiter(..., ' ') extracts the leading numeric token. If that is unavailable, Int64.From safely converts the cell, defaulting null or invalid entries to 0 (Int64.Type).
2. Boolean Flag Casting: For both 'is_new_construction' and 'is_verified', the input is lowercased and tested against List.Contains({'true', 'yes', '1'}, ...). If matched, it returns true; otherwise, it resolves cleanly to false.
This guarantees true binary flags ('type logical') that consume minimal memory in the VertiPaq engine.

```
// 11. تنظيف عدد الصور والأعمدة المنطقية
#"Cleaned Text(image count)" = Table.TransformColumns(#"Cleaned Coordinates(lat & lon)", {
    {"image_count", each
        if _ = null then 0
        else
            let s = Text.BeforeDelimiter(Text.Trim(Text.From(_)), " ") in
            try Int64.From(s) otherwise (try Int64.From(_) otherwise 0),
        Int64.Type},
    {"is_new_construction", each
        if _ = null then false
        else List.Contains({"true", "yes", "1"}, Text.Lower(Text.Trim(Text.From(_)))),
        type logical},
    {"is_verified", each
        if _ = null then false
        else List.Contains({"true", "yes", "1"}, Text.Lower(Text.Trim(Text.From(_)))),
        type logical}
}),
```

> **Key Analytical Takeaway:** Data Typing: Converts annotations into clean integers and maps mixed truthy values ('yes', '1', 'true') to native Power BI booleans.

Step 12: Geographic Hierarchy Feature Engineering (Compound & District)

Applied Power Query Steps: #"Added Custom(compound neighborhood)", #"Added Custom(district)"

Problem in Raw Data: The raw dataset provided only a single concatenated text column 'location_full' (e.g., 'Palm Hills, 6th of October City, Giza' or 'Mountain View iCity, New Cairo, Cairo'). Analysts could not group properties by compound project or drill down from Governorate -> District -> Compound, severely limiting dashboard analytical depth.

Transformation Logic & Implementation: Two new granular dimensions are engineered using Table.AddColumn:
1. compound_neighborhood: Extracts the primary segment preceding the first comma using Text.BeforeDelimiter([location_full], ','). This reliably isolates specific compound projects (e.g., 'Palm Hills', 'Madinaty', 'Mivida', 'Zed') or primary neighborhood streets.
2. district: Decomposes [location_full] into a cleaned list of comma-separated parts. If 3 or more segments exist (Compound, District, City), it dynamically extracts cleanParts{cnt - 2} to capture the municipal district (e.g., 'New Cairo', 'Sheikh Zayed City', 'Maadi'). If fewer segments exist, it falls back gracefully to the first component.
This creates a complete 3-level Star Schema geographic hierarchy: City -> District -> Compound.

```
// 12. استخراج الحي والكمبوند من العنوان (Location Hierarchy)
#"Added Custom(compound neighborhood)" = Table.AddColumn(#"Cleaned Text(image count)",
"compound_neighborhood", each
    let
        s = [location_full],
        beforeComma = if s = null or s = "" then null else Text.Trim(Text.BeforeDelimiter(s, ","))
    in
        if beforeComma = null or beforeComma = "" then (if s = null or s = "" then null else
Text.Trim(s)) else beforeComma,
    type text
),

#"Added Custom(district)" = Table.AddColumn(#"Added Custom(compound neighborhood)", "district", each
    let
        s = [location_full],
        parts = if s = null or s = "" then {} else List.Transform(Text.Split(s, ","), Text.Trim),
        cleanParts = List.Select(parts, each _ <> ""),
        cnt = List.Count(cleanParts)
    in
        if cnt >= 3 then cleanParts{cnt - 2}
        else if cnt = 2 then cleanParts{0}
        else if cnt = 1 then cleanParts{0}
        else null,
    type text
),
```

> **Key Analytical Takeaway:** Feature Engineering: Unlocks a robust 3-tier geographic drill-down hierarchy: Governorate (City) → District → Compound / Project.

Step 13: Amenities Delimiter Harmonization & Feature Count Calculation

Applied Power Query Steps: #"Standardized Delimiters(amenities)", #"Added Custom(amenities count)"

Problem in Raw Data: Listing amenities were formatted using conflicting arbitrary delimiters across different scraping sources (e.g., 'Balcony; Security|Private Garden/Pool'). Slicers or word clouds could not separate individual amenities. Furthermore, there was no numeric metric to measure property luxury level based on the total number of bundled amenities.

Transformation Logic & Implementation: 1. Delimiter Harmonization: The transformation sequentially replaces semicolons ';', pipes '|', and slashes '/' with standard commas ','. The string is split into individual items, trimmed of whitespace, filtered to remove empty elements, and recombined with uniform ', ' separation. If no amenities are listed, it defaults to 'None'.
2. Amenities Count Engineering: Table.AddColumn calculates 'amenities_count'. If [amenities] is 'None' or null, it assigns 0. Otherwise, it counts the elements in the normalized list using List.Count(Text.Split([amenities], ', ')).
This provides a continuous quantitative feature (Int64.Type) that correlates strongly with property valuation.

```
// 13. توحيد فواصل الخدمات وحساب عددها
#"Standardized Delimiters(amenities)" = Table.TransformColumns(#"Added Custom(district)", {
    {"amenities", each
        if _ = null or Text.Trim(Text.From(_)) = "" then "None"
        else
            let
                raw = Text.Trim(Text.From(_)),
                r1 = Text.Replace(raw, ";", ","),
                r2 = Text.Replace(r1, "|", ","),
                r3 = Text.Replace(r2, "/", ","),
                items = List.Select(List.Transform(Text.Split(r3, ","), Text.Trim), each _ <> ""),
                cleanText = if List.IsEmpty(items) then "None" else Text.Combine(items, ", ")
            in
                cleanText,
        type text}
}),

#"Added Custom(amenities count)" = Table.AddColumn(#"Standardized Delimiters(amenities)",
"amenities_count", each
    if [amenities] = "None" or [amenities] = null then 0
    else List.Count(Text.Split([amenities], ", ")),
    Int64.Type
),
```

> **Key Analytical Takeaway:** Quantitative Luxury Metric: Normalizes varied delimiters and creates an integer 'amenities_count' metric for price correlation modeling.

Step 14: Broker Text Formatting & Payment Method Standardization

Applied Power Query Steps: #"Capitalized Each Word(broker name)"

Problem in Raw Data: Broker names, emails, furnishing specifications, and payment terms suffered from inconsistent letter casing (e.g., all uppercase broker names, mixed-case email addresses, and unformatted payment tokens like 'cash_installments'). This created an unprofessional visual appearance in report cards and broker leaderboards.

Transformation Logic & Implementation: Multiple presentation columns are standardized in a single Table.TransformColumns step:
• broker_name: Formatted using Text.Proper to capitalize each word cleanly.
• broker_email: Lowercased using Text.Lower to ensure RFC-compliant, consistent email formatting.
• broker_id & completion_status: Stripped of any residual padding via Text.Trim.
• furnished: Standardized to uppercase (Text.Upper) for distinct visual badge displays ('FURNISHED', 'UNFURNISHED').
• listing_level: Formatted using Text.Proper ('Premium', 'Standard').
• payment_method: Standardized by mapping 'cash' -> 'Cash', 'installments' -> 'Installments', 'cash_installments' -> 'Cash & Installments', with fallback to Text.Proper.
This ensures polished, executive-ready presentation in Power BI tables and cards.

```
// 14. تنسيق أسماء وإيميلات الوسطاء وطريقة الدفع
#"Capitalized Each Word(broker name)" = Table.TransformColumns(#"Added Custom(amenities count)", {
    {"broker_name", Text.Proper, type text},
    {"broker_email", Text.Lower, type text},
    {"broker_id", Text.Trim, type text},
    {"furnished", Text.Upper, type text},
    {"listing_level", Text.Proper, type text},
    {"completion_status", Text.Trim, type text},
    {"payment_method", each
        let p = Text.Lower(Text.Trim(Text.From(_))) in
        if p = "cash" then "Cash"
        else if p = "installments" then "Installments"
        else if p = "cash_installments" then "Cash & Installments"
        else Text.Proper(p),
        type text}
}),
```

> **Key Analytical Takeaway:** Executive Presentation: Applies title casing to broker names, standardizes emails, and normalizes financial payment terms.

Step 15: Logical Column Reordering by Business Domain

Applied Power Query Steps: #"Reordered Columns"

Problem in Raw Data: After feature engineering and data transformations, columns were scattered in arbitrary order across the table. In a BI development environment, developers waste substantial time searching for related

fields (e.g., finding that 'compound_neighborhood' was placed at the very end of the table while 'location_full' was near the front).

Transformation Logic & Implementation: Table.ReorderColumns rearranges all 31 cleaned attributes into 8 intuitive business clusters:
1. Primary Key: listing_id
2. Property Classification: category, listing_type, property_type, completion_status
3. Financial Metrics: price_egp, price_period
4. Geographic Hierarchy: location_full, compound_neighborhood, district, city, lat, lon
5. Physical Specifications: bedrooms, bathrooms, area_value, furnished, listing_level
6. Features & Amenities: is_new_construction, is_verified, listed_date, image_count, amenities, amenities_count, payment_method
7. Broker Details: broker_id, broker_name, broker_email
8. Transaction Lifecycle: status, sold_price, sale_date
This logical clustering dramatically improves report authoring workflow and data model comprehension.

```
// 15. إعادة ترتيب الأعمدة بشكل منطقي
#"Reordered Columns" = Table.ReorderColumns(#"Capitalized Each Word(broker name)",{
    "listing_id", "category", "listing_type", "property_type", "completion_status",
    "price_egp", "price_period", "location_full", "compound_neighborhood", "district", "city",
    "lat", "lon", "bedrooms", "bathrooms", "area_value", "furnished", "listing_level",
    "is_new_construction", "is_verified", "listed_date", "image_count",
    "amenities", "amenities_count", "payment_method",
    "broker_id", "broker_name", "broker_email",
    "status", "sold_price", "sale_date"
}),
```

> **Key Analytical Takeaway:** Developer Ergonomics: Groups all 31 fields into 8 intuitive business clusters, speeding up visual design and measure authoring.

Step 16: Final Strict Data Type Enforcement & Schema Locking

Applied Power Query Steps: #"Changed Type"

Problem in Raw Data: Without explicit data typing, Power Query columns remain typed as generic 'any'. This forces Power BI's VertiPaq storage engine to defer columnar encoding decisions, degrading in-memory compression ratios and causing DAX calculations to incur type coercion overhead during query execution.

Transformation Logic & Implementation: Table.TransformColumnTypes explicitly assigns strict, native Power BI data types to all 31 attributes:
• type text: listing_id, category, listing_type, property_type, completion_status, price_period, location_full, compound_neighborhood, district, city, furnished, listing_level, amenities, payment_method, broker_id, broker_name, broker_email, status.
• type number: price_egp, area_value, lat, lon, sold_price.
• Int64.Type: bedrooms, bathrooms, image_count, amenities_count.
• type logical: is_new_construction, is_verified.
• type date: listed_date, sale_date.

Locking data types as the final step maximizes VertiPaq dictionary and bit-packed compression, reducing report memory footprint and guaranteeing blazing-fast dashboard interactivity.

```
// 16. تعيين أنواع البيانات النهائية لكل عمود
#"Changed Type" = Table.TransformColumnTypes(#"Reordered Columns",{
    {"listing_id", type text},
    {"category", type text},
    {"listing_type", type text},
    {"property_type", type text},
    {"completion_status", type text},
    {"price_egp", type number},
    {"price_period", type text},
    {"location_full", type text},
    {"compound_neighborhood", type text},
    {"district", type text},
    {"city", type text},
    {"lat", type number},
    {"lon", type number},
    {"bedrooms", Int64.Type},
    {"bathrooms", Int64.Type},
    {"area_value", type number},
    {"furnished", type text},
    {"listing_level", type text},
    {"is_new_construction", type logical},
    {"is_verified", type logical},
    {"listed_date", type date},
    {"image_count", Int64.Type},
    {"amenities", type text},
    {"amenities_count", Int64.Type},
    {"payment_method", type text},
    {"broker_id", type text},
    {"broker_name", type text},
    {"broker_email", type text},
    {"status", type text},
    {"sold_price", type number},
    {"sale_date", type date}
})
in
    #"Changed Type"
```

> **Key Analytical Takeaway:** Performance Optimization: Locking strict data types enables VertiPaq columnar compression, minimizing memory usage and boosting DAX execution.

3. Final Data Model Schema & Attribute Dictionary

Upon successful completion of the 16 ETL pipeline steps, the resulting dataset consists of 246,077 clean, deduplicated records structured across 31 strictly typed attributes. The table below details the final schema specification, including data types, business descriptions, and their analytical roles within the Power BI Star Schema data model.

Attribute Name	Power BI Data Type	Analytical Role	Business Description
`listing_id`	Text	**Primary Key (1:1)**	Unique property listing identifier for table joins.
`category`	Text	**Dimension / Filter**	Standardized transaction category ('Rent' or 'Sale').
`listing_type`	Text	**Dimension**	Original listing market classification.
`property_type`	Text	**Dimension / Slicer**	Harmonized property typology (12 canonical types).
`completion_status`	Text	**Dimension**	Construction completion state ('Under Construction', 'Ready').
`price_egp`	Decimal Number	**Fact Metric**	Standardized asking price in Egyptian Pounds (EGP).
`price_period`	Text	**Dimension / Filter**	Pricing frequency ('Monthly' for rent, 'Total' for sale).
`location_full`	Text	**Descriptive Attribute**	Complete raw location string from crawl source.
`compound_neighborhood`	Text	**Dimension / Hierarchy**	Engineered Compound/Project name (Level 3 Hierarchy).
`district`	Text	**Dimension / Hierarchy**	Engineered municipal District (Level 2 Hierarchy).
`city`	Text	**Dimension / Hierarchy**	Standardized Egyptian Governorate (Level 1 Hierarchy).
`lat`	Decimal Number	**Geospatial Coordinate**	Corrected latitude for map pin plotting (22°–32° N).
`lon`	Decimal Number	**Geospatial Coordinate**	Corrected longitude for map pin plotting (25°–36° E).
`bedrooms`	Whole Number	**Fact / Specification**	Total bedroom count.
`bathrooms`	Whole Number	**Fact / Specification**	Total bathroom count.
`area_value`	Decimal Number	**Fact Metric**	Property floor area measured in square meters (m^2).
`furnished`	Text	**Dimension**	Furnishing status ('FURNISHED', 'UNFURNISHED').
`listing_level`	Text	**Dimension**	Brokerage tier ('Standard', 'Premium').
`is_new_construction`	Logical (Boolean)	**Flag / Slicer**	True if primary new construction; False if resale.
`is_verified`	Logical (Boolean)	**Flag / Slicer**	True if broker/listing documents are officially verified.

Attribute Name	Power BI Data Type	Analytical Role	Business Description
`listed_date`	Date	**Time Dimension Link**	Market launch date; joins to central Date table.
`image_count`	Whole Number	**Fact Metric**	Clean integer count of property photos.
`amenities`	Text	**Descriptive / Cloud**	Clean comma-separated listing of property features.
`amenities_count`	Whole Number	**Engineered Metric**	Quantitative luxury index (count of bundled amenities).
`payment_method`	Text	**Dimension**	Standardized payment terms ('Cash', 'Installments', etc.).
`broker_id`	Text	**Dimension Key**	Unique identifier for the listing real estate broker.
`broker_name`	Text	**Dimension / Slicer**	Clean proper-cased real estate agent / agency name.
`broker_email`	Text	**Descriptive Attribute**	Normalized lowercase broker contact email.
`status`	Text	**Dimension / Slicer**	Current status ('Available', 'Sold', 'Rented', 'Off Market').
`sold_price`	Decimal Number	**Fact Metric**	Realized closing transaction price (strictly null if unsold).
`sale_date`	Date	**Time Dimension Link**	Transaction completion date (strictly null if unsold).

4. Power BI Modeling & DAX Implementation Guidelines

With data cleaning completed at the Power Query level, the table can now serve as the central Fact table (or be split into a Fact and Dimension model) inside Power BI Desktop. The following modeling recommendations ensure optimal performance and DAX accuracy:

1. Active vs. Closed Transactions: Always filter by [status] = "Available" when calculating active listing metrics (e.g., Total Active Inventory, Current Asking Price per m^2). Conversely, use [status] IN {"Sold", "Rented"} when calculating sales revenue or transaction volume.
2. Central Date Table: Connect [listed_date] and [sale_date] to an independent DAX Calendar table. Maintain the active relationship on [listed_date] and an inactive relationship on [sale_date] activated via USERELATIONSHIP(Calendar[Date], RealEstate[sale_date]) in specialized sales measures.
3. Price per Square Meter: Formulate dynamic DAX measures rather than calculated columns:
   Price_Per_SQM = DIVIDE(SUM(RealEstate[price_egp]), SUM(RealEstate[area_value]), BLANK())
4. Compound Drill-Down: Build a native Power BI hierarchy using [city] -> [district] -> [compound_neighborhood] to provide intuitive drill-through capabilities across map and matrix visuals.